Day 1 - Linux Fundamentals

Why Linux Matters in DevOps

Linux is the operational backbone of modern infrastructure.

Most DevOps tools and platforms run on Linux:

- CI/CD runners
- Docker containers
- Kubernetes clusters
- Cloud virtual machines
- Monitoring systems
- Automation servers

Important Point

> Kubernetes itself is Linux-centric.

Linux Mental Model

Linux is built around:

- Files
- Processes
- Permissions
- Networking
- Services

Core Linux Philosophy

> In Linux, almost everything is treated like a file.

Examples:

Resource	Linux Representation
Hard disk	File/device
Logs	Files
Process information	`/proc`
Devices	Files

Resource	Linux Representation
Network sockets	Files

Linux Architecture

```
Applications
    ↓
Shell (Bash)
    ↓
Linux Kernel
    ↓
Hardware
```

Components

Component	Purpose
Kernel	Core OS engine
Shell	Command interpreter
Filesystem	Data organization
Processes	Running programs
systemd	Service manager
Networking Stack	Communication

Important Linux Concepts

Process

A running program.

Examples:

- nginx
- java
- tomcat
- docker

Service

A background managed process.

Examples:

- sshd
- nginx
- docker

Filesystem

Storage structure used by Linux.

Permissions

Controls security and access.

Process Lifecycle

```
Program on Disk
      ↓
Executed
      ↓
Running Process
      ↓
Uses CPU + Memory
      ↓
Terminates
```

Zombie Process

A dead process that has completed execution but still exists because the parent process has not cleaned it.

Essential Linux Directories

Linux starts from:

```
/
```

This is called the:

> Root Filesystem

Important Directories

Directory	Purpose
`/home`	User files
`/etc`	Configuration files
`/var/log`	System and application logs
`/tmp`	Temporary files
`/bin`	Essential commands
`/proc`	Process/kernel information
`/dev`	Device files
`/opt`	Optional software
`/usr`	User binaries/libraries
`/sbin`	System administration binaries

Important DevOps Files

DNS Configuration

```
/etc/resolv.conf
```

Hosts File

```
/etc/hosts
```

Used for:

- Local hostname mapping

- Internal testing
- DNS overrides

User Accounts

```
/etc/passwd
```

Filesystem Mounts

```
/etc/fstab
```

Understanding /proc

`/proc` is a virtual filesystem.

> These are not real files on disk.

Used for kernel and process information.

Examples

CPU Information

```
/proc/cpuinfo
```

Memory Information

```
/proc/meminfo
```

Process Information

```
/proc/<PID>
```

Example:

```
/proc/1234
```

Basic Linux Commands

1. Check Current Shell

```
echo $SHELL
```

Example Output:

```
/bin/bash
```

---

2. Print Text

```
echo "Hello DevOps"
```

---

3. List Files

```
ls
```

Detailed format:

```
ls -lah
```

Important Flags

Option	Meaning
`-l`	Long format
`-a`	Hidden files
`-h`	Human readable

---

4. Present Working Directory

```
pwd
```

Displays current directory.

---

5. Change Directory

```
cd /var/log
```

Go back:

```
cd ..
```

Go to home:

```
cd ~
```

Directory Operations

Create Directory

```
mkdir new-directory
```

Create Nested Directories

```
mkdir -p /tmp/asia/india/bangalore
```

Remove Directory

```
rm -r mydir
```

⚠️ Be careful with recursive delete.

File Operations

Create File

```
touch newfile.txt
```

Add Content to File

```
cat > newfile.txt
```

Press:

```
CTRL + D
```

To save and exit.

View File Content

```
cat newfile.txt
```

Copy File

```
cp newfile.txt copy-file.txt
```

Copy Directory

```
cp -r mydir /tmp/my-dir
```

Move or Rename File

```
mv old.txt new.txt
```

Delete File

```
rm file.txt
```

Delete Directory Recursively

```
rm -rf folder
```

Important Production Warning

⚠️ `rm -rf` is extremely dangerous.

Always verify:

- Current path
- Target directory
- Environment (Dev/Test/Prod)

Viewing File Content

Entire File

```
cat file.txt
```

Large Files

```
less file.txt
```

Quit:

```
q
```

First Lines

```
head file.txt
```

Last Lines

```
tail file.txt
```

Live Log Monitoring

```
tail -f app.log
```

Common Use Cases

- Application debugging
- Production incidents
- Deployment monitoring
- Kubernetes pod logs

Real Example

```
tail -f /var/log/syslog
```

---

grep - Search Inside Files

Search Text

```
grep ERROR app.log
```

Case Insensitive Search

```
grep -i error app.log
```

Recursive Search

```
grep -r database /etc
```

Common DevOps Use Cases

- Log analysis
- Troubleshooting
- CI/CD debugging
- Monitoring analysis

Process Management

View Processes

```
ps -ef
```

Meaning

Option	Meaning
`-e`	All processes
`-f`	Full format

Displays

- User
- PID
- Parent PID
- Start time
- Command

Difference Between ps -ef and ps aux

Command	Style
`ps -ef`	UNIX style options
`ps aux`	BSD style options

Both show process information.

Filter Processes

Find nginx Process

```
ps -ef | grep nginx
```

Find Java Process

```
ps -ef | grep java
```

Find Kubernetes Related Processes

```
ps -ef | grep kube
```

Process Tree

```
pstree
```

Shows:

- Parent-child hierarchy
- Process relationships

Real-Time Monitoring

top

```
top
```

htop

```
htop
```

`htop` is an improved interactive version.

Important Monitoring Metrics

Metric	Meaning
CPU	Processor usage
MEM	Memory usage
PID	Process ID
USER	Process owner

Kill Processes

Graceful Kill

```
kill PID
```

Example:

```
kill 1234
```

Force Kill

```
kill -9 PID
```

Example:

```
kill -9 1234
```

⚠️ Dangerous command.

Why?

Because cleanup may fail:

- Data corruption possible
- Connections may break
- Transactions may fail
- Graceful shutdown skipped

CPU vs Memory Understanding

CPU

CPU measures active computation.

High CPU Indicates

- Infinite loops
- Heavy calculations
- Excessive threads

Symptoms

- System becomes slow
- High load average

Memory

Memory stores process data in RAM.

High Memory Usage Indicates

- Memory leaks
- Heavy caching
- Large datasets
- Inefficient applications

Possible Outcomes

- OOM kills
- Crashes

- Memory exhaustion

Memory and Disk Commands

Memory Usage

```
free -h
```

Shows:

- Total
- Used
- Free
- Available

Disk Usage

```
df -h
```

Checks mounted filesystems.

Directory Size

```
du -sh *
```

System Load

```
uptime
```

Important Note

Load average:

> Runnable/waiting processes over time

It is NOT CPU percentage.

Linux Troubleshooting Flow

```
Symptom
   ↓
Observe
   ↓
Identify failing layer
   ↓
Check logs
   ↓
Validate resources
   ↓
Find root cause
   ↓
Fix safely
```

Logs and Troubleshooting

Important Log Files

File	Purpose
`/var/log/syslog`	System logs
`/var/log/auth.log`	Authentication logs
`/var/log/kern.log`	Kernel logs

View Logs

```
less /var/log/syslog
```

Search Errors

```
grep ERROR /var/log/syslog
```

journalctl (VERY Important)

Modern Linux systems use:

> systemd journal

View Logs

```
journalctl
```

Last 50 Logs

```
journalctl -n 50
```

Live Logs

```
journalctl -f
```

Service Specific Logs

```
journalctl -u ssh
```

Why journalctl Matters

Heavily used in:

- Linux servers
- Kubernetes nodes
- Cloud VMs
- CI/CD runners

Networking Commands

Check IP Address

```
ip a
```

DNS Test

```
nslookup google.com
```

Connectivity Test

```
ping google.com
```

Check Open Ports

```
ss -tulnp
```

Why Important?

Used to:

- Verify listening services
- Check open ports
- Troubleshoot connectivity

---

ss vs netstat

Modern Linux prefers:

```
ss
```

Instead of:

```
netstat
```

Why?

- Faster
- More efficient
- Better socket inspection

Linux Permissions

View Permissions

```
ls -l
```

Example:

```
-rwxr-xr--
```

Permission Breakdown

Symbol	Meaning
r	Read
w	Write
x	Execute

Structure

```
Owner | Group | Others
```

Change Permission

```
chmod +x script.sh
```

Makes script executable.

Change Ownership

```
chown user:user file.txt
```

User Management

Current User

```
whoami
```

User Details

```
id
```

Shows:

- UID
- GID
- Groups

Switch User

```
su username
```

SSH Login

```
ssh username@<ip-address>
```

Example:

```
ssh devops@192.168.1.10
```

---

sudo

Used for elevated privileges.

Example:

```
sudo ls /root
```

Configuration file:

```
/etc/sudoers
```

Download Files

curl

```
curl URL -O
```

Example:

```
curl https://example.com/file.zip -O
```

---

wget

```
wget URL -O filename
```

Example:

```
wget https://example.com/file.zip -O app.zip
```

VI / VIM Editor Basics

Open File

```
vi file.txt
```

Modes in VI

Command Mode

Default mode.

Used for:

- Navigation
- Copy
- Delete

Insert Mode

Press:

```
i
```

Used to:

- Add content

- Modify content

Exit insert mode:

```
ESC
```

Delete Operations

Delete Character

```
x
```

Delete Line

```
dd
```

Copy and Paste in VI

Copy Line

```
yy
```

Paste

```
p
```

Save and Quit

Save

```
:w
```

Quit Without Saving

```
:q!
```

Save and Exit

```
:wq
```

Search Text in VI

Search Word

```
/of
```

Highlights occurrences in file.

DevOps Operational Mindset

As a DevOps engineer always ask:

Question	Why
Is process running?	Service health
CPU high?	Performance/scaling issue
Memory leaking?	Application issue
Disk full?	Operational issue
Logs showing errors?	Root cause analysis
Port listening?	Connectivity issue

Troubleshooting Order (VERY Important)

Always investigate in this order:

```
Infrastructure
   ↓
Operating System
   ↓
Network
   ↓
Application
   ↓
Dependencies (DB/API/Cache)
```

Mini Hands-On Practice

Exercise 1

```
cd /tmp
mkdir week1
cd week1
touch file1.txt file2.txt file3.txt
```

Exercise 2

```
ls -lah
```

Exercise 3

```
echo "DevOps Learning" > notes.txt
cat notes.txt
```

Exercise 4

Open another terminal and run:

```
top
```

Observe:

- CPU
- Memory
- Processes

Exercise 5

```
tail -f /var/log/syslog
```

Observe live logs.

Interview Questions

Scenario 1

Production Linux server suddenly becomes slow.

What should you check?

- CPU usage
- Memory usage
- Disk usage
- Running processes
- Logs
- Network latency

Commands:

```
top
free -h
df -h
ps -ef
journalctl -f
```

Scenario 2

Application team says server has memory issue.

First Checks

```
free -h
ps -ef
htop
```

Look for:

- High memory processes
- Memory leaks
- OOM kills

Scenario 3

Disk usage reached 100%.

Troubleshooting Steps

```
df -h
du -sh *
```

Check:

- Large logs
- Old backups
- Temporary files

Important Commands to Memorize

```
ps -ef
top
htop
tail -f
grep
journalctl
ss -tulnp
```

```
free -h
df -h
ip a
```

Key Takeaways

- Linux is foundational to DevOps.
- Logs are the first place to check during incidents.
- Learn process monitoring deeply.
- Understand permissions carefully.
- Networking basics are mandatory.
- Troubleshooting mindset matters more than memorizing commands.

Suggested Practice for Tomorrow

Practice these repeatedly:

- File operations
- Process monitoring
- Log analysis
- Permissions
- Networking commands
- VI editor

The goal is:

> Become comfortable operating Linux without fear.

Reference Notes

Prepared using uploaded Linux notes and command practice sessions. filecite turn0file0 filecite turn0file1 filecite turn0file2